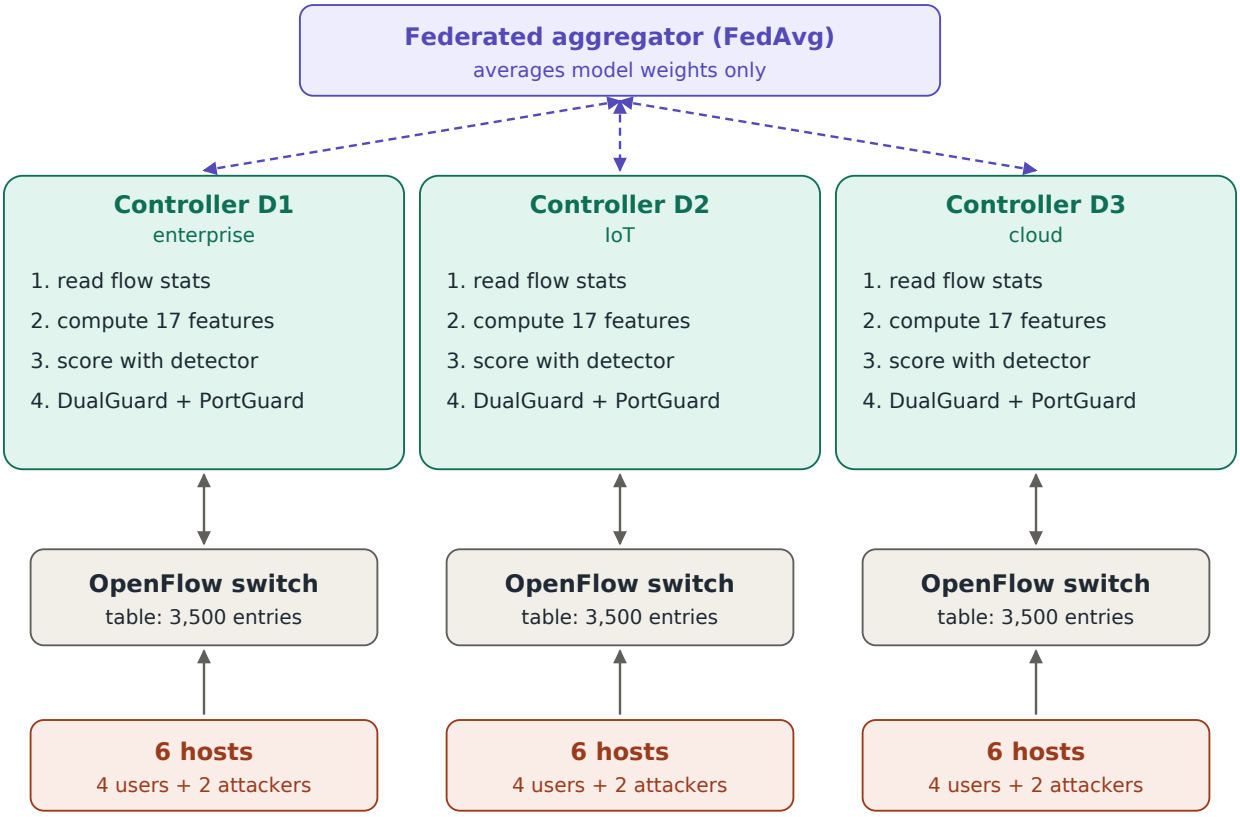
FL-DualGuard, step by step

How our multi-controller defense against low-rate flow-table overflow (LFTO) attacks is built, explained in plain language.

How to read this guide. Steps 1–3 explain the problem and the test environment. Steps 4–9 follow one second of the controller's life, from reading the switch to acting on it. Steps 10–11 explain how we measure and test fairly. The last pages map every step to the code and list the results so far.

The whole system on one page



Dashed purple: model weights travel between controllers; raw traffic never leaves its domain.

Figure 1. Three network domains, each with its own controller and switch. Controllers learn together by sharing model weights, never traffic.

- **Domains.** Three networks with different traffic: an enterprise (D1), an IoT edge with many periodic sensor flows (D2), and a cloud front-end with flash crowds (D3).
- **Switch.** Each domain has an OpenFlow switch whose flow table holds 3,500 entries. One entry is needed per active connection.
- **Controller.** Once a second it reads the table, scores every entry with a small neural network, and decides what to do.
- **Aggregator.** During training, the controllers average their neural-network weights (federated learning), so each one learns attacks it never saw itself.

Step 1 — The problem: filling the table slowly

A switch forwards a packet quickly only if a matching entry is in its flow table. The table is small (3,500 entries here), and an entry is removed after 10 seconds without traffic (the idle timeout).

A low-rate attacker opens thousands of fake connections and sends one tiny packet on each of them every 5–9.5 seconds. That is just often enough to keep every fake entry alive. The attack adds about 100 new fake entries every few seconds until each attacker holds 1,700. The table fills up, and new connections from real users are refused, even though the attacker sends very little traffic and looks harmless to a bandwidth monitor.

We use the same attack recipe as the DMS-BTT paper (Tang et al., 2025) and add harder variants:

Variant	What it does	Seen in training?
A1–A3	The three DMS-BTT settings (refresh every 5–9.5 s, 75/100/125 new flows per burst, 1,700 flows per attacker)	A1 by D1, A2 by D2, A3 by D3
A4 mimic	Copies normal packet sizes and sends 1–3 packets per refresh	D1
A5 spoof	Fakes a random source IP on every flow	D2
A6 slow-fill	Very slow, spoofed, mimicking	Never (a "zero-day")
A7 pulse	Attacks for 60 s, pauses 30 s, repeats	D3
Fast-refresh (step 4)	Refreshes every 0.25–1 s to defeat demotion	Never

Step 2 — Generating traffic

Code: `loftbench/traffic.py`, `config.py`, `ports.py`

- **Normal traffic** is generated packet by packet from six kinds of flows: DNS look-ups, short web flows ("mice"), large downloads ("elephants"), periodic keep-alives and sensors, interactive sessions, and flash crowds. Each domain mixes them differently.
- **Attack traffic** follows the recipe in Step 1, with the variant's settings.
- **Labels come from where a packet came from**, never from its features, so the detector cannot learn a shortcut.
- **Paired runs.** The same random seed always gives the same normal traffic, so every defense is tested on exactly the same users.
- **Ports (new in step 4).** As in the DMS-BTT topology, each host has its own switch port: four normal hosts and two attackers. A spoofer can fake its IP address but not the port it is plugged into.

Step 3 — The switch simulator

Code: `loftbench/sim.py` (unit-tested in `tests/`)

The simulator replays the traffic in steps of 0.1 seconds and behaves like an OpenFlow switch:

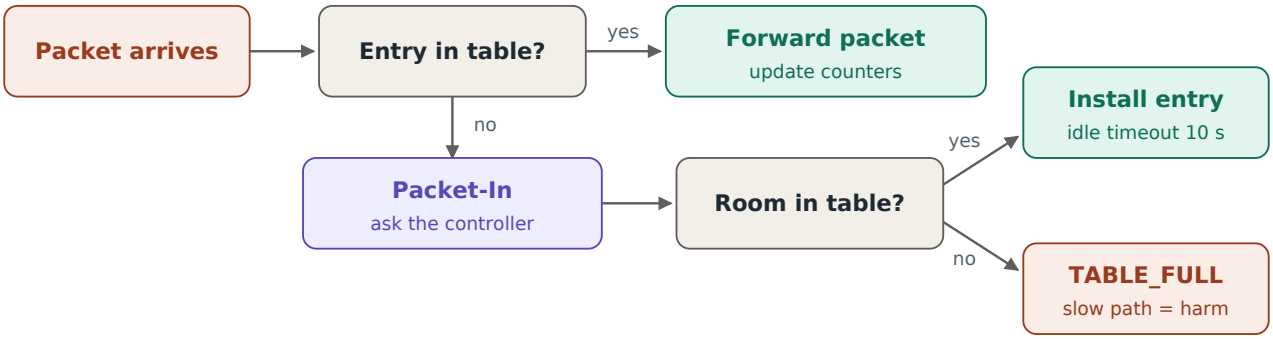


Figure 2. What happens to each packet in each 0.1 s step.

- Entries idle for longer than their timeout are removed first.
- A packet that matches an entry is forwarded and updates the entry's counters.
- A packet with no entry goes to the controller (a Packet-In). If there is room, the controller installs an entry; if not, the switch reports TABLE_FULL and the packet takes the slow path through the controller. That is the harm the attack causes.
- Every second, the controller reads the table and may delete or demote entries (Steps 4–9).

The same logic is implemented for a real Ryu controller and Mininet/Open vSwitch in `testbed/`; those runs need administrator rights and have not been run yet.

Steps 4–9 — One second in the life of a controller

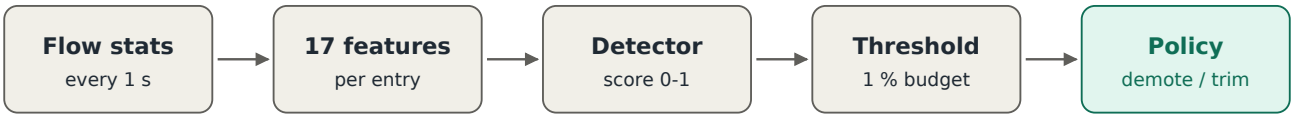


Figure 3. The controller's loop, repeated every second.

Step 4 — Read the flow statistics

The controller asks the switch for every entry's packet count, byte count, age and source address. That is all a standard OpenFlow 1.3 controller can see, so the method works on real switches.

Step 5 — Compute 17 features per entry

Code: `loftbench/features.py`

Group	Features	What they capture
Flow (11)	packets, bytes, age, mean packet size, packet rate, time between packets, new packets since last poll, polls since last activity, share of polls active, regularity of gaps, mean gap	Does this entry behave like a fake flow kept barely alive?
Source (3)	entries from the same source, new entries from it in the last 10 s, its share of tiny flows	Is one source opening many small flows?

Group	Features	What they capture
Context (3)	table occupancy, install rate, average time between packets across the table	Is the table under pressure right now?

Step 6 — Score each entry with the detector

Code: `loftbench/models.py`

The detector is a small neural network (17 inputs → 32 hidden units → 1 output) that gives every entry a score between 0 (normal) and 1 (attack).

Federated training (FedAvg). Each controller trains on its own traffic, which contains only two attack variants. Over 30 rounds, the controllers send their weights to the aggregator, which averages them and sends the result back. The shared model therefore recognises all variants seen by any controller, while no traffic records leave a domain. A small model also makes this cheap: about 2.4 KB per controller per round.

Why it matters. In step 3 of the revision we tested the alternatives under the same policy: learning alone harms 7.9 % of normal flows, learning alone plus the public DMS-BTT attack settings 5.8 %, and federation 1.5 %. Pooling all raw data gives 1.7 %, so federation matches it without sharing data.

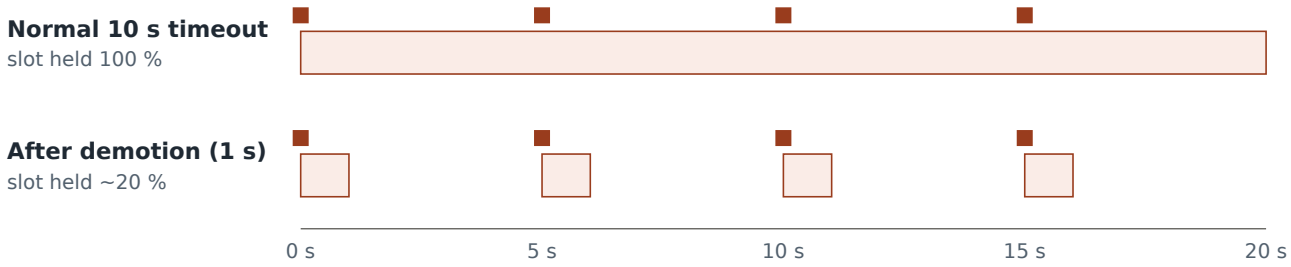
Step 7 — Turn scores into decisions with a false-alarm budget

A score needs a cut-off. Each controller sets its own cut-off on its own normal validation traffic so that at most **1 % of normal entries** would be flagged. This is the calibration budget. Different domains get different cut-offs, because their normal traffic differs.

Step 8 — DualGuard: demote, don't delete

Code: `loftbench/defenses.py`, class `DualGuard`

- **Act only under pressure.** If the table is less than 70 % full, do nothing.
- **Demote flagged entries.** A flagged entry is removed, and when its next packet arrives it is reinstalled with a **1-second** idle timeout instead of 10 seconds.
- **Why this hurts the attacker.** A fake flow refreshed every 5 s now occupies its slot only about 1 s in 5 (Figure 4), so the table drains.
- **Why it is gentle on users.** A real flow wrongly flagged keeps working: an active flow sends faster than once a second and never notices; a paused one pays one extra round trip to the controller.



Square = one attacker packet (every 5 s). Bar = time the entry occupies a table slot.

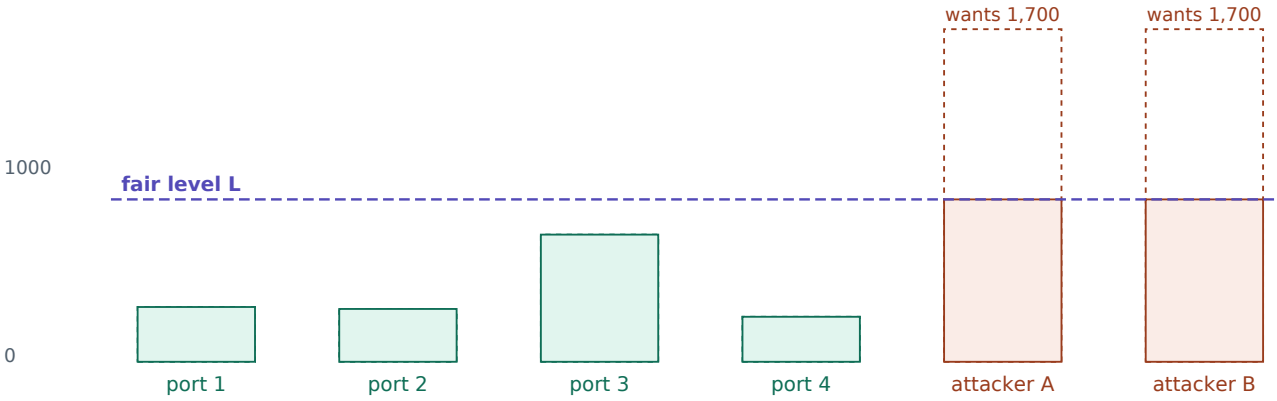
Figure 4. Demotion shrinks how long a slow attacker holds each table slot.

Step 9 — PortGuard: a fair share per port (step 4)

Code: `loftbench/defenses.py`, class `PortGuard`

The gap it closes. An attacker who refreshes faster than once a second keeps even demoted entries alive, and the detector recognises only about 2 % of its entries because they look like busy normal flows. DualGuard alone then fails (62 % of normal flows harmed in our check).

The idea. When the table is under pressure, every switch port gets a fair share of it, computed so that small users keep everything they need and only the biggest consumers are cut back (max-min fairness):



Solid bar: entries kept. Dashed outline: what the port asked for.

Figure 5. Ports below the fair level keep all their entries; the attackers are cut back to it.

- Ports above their share are trimmed back, removing their most suspicious entries first (the detector still chooses which ones).
- Those ports are refused new entries until they fall below their share. The controller counts its own installs, so the limit holds between polls.
- **Guarantee.** Every honest port keeps at least its fair share, however fast the attacker refreshes, whatever packet sizes it uses and whatever IP addresses it fakes.
- **Limit.** If the attacker controls a host on every port, it gets a fair share on every port, and PortGuard cannot single it out. The test measures this case on purpose.

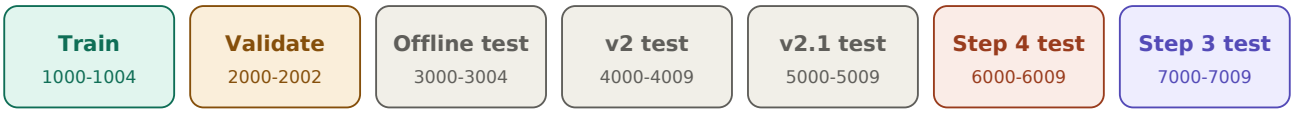
Test result (fresh seeds, attackers on their own ports). PortGuard alone keeps harm at about 6 % for every attacker speed from one refresh per 0.25 s to one per 9 s, where every learned defense harms 15–81 %. Combined with DualGuard it does worse (36 % against fast attackers): with the table held near full, DualGuard keeps demoting normal entries, and PortGuard hands the freed slots to the attackers. Making the two cooperate is the next step (4b).

Step 10 — Measuring what users feel

Metric	Meaning
Normal flows harmed (primary)	Share of normal connections that were refused an entry, or removed while still active (their next packet arrived before the entry would have expired anyway)
Malicious occupancy	Average share of the table held by attack entries
Overflow time	Seconds the table was completely full during the attack
Packet-In / TABLE_FULL	Load on the controller and the control channel

Statistics. Each (domain, seed) pair is one independent cluster (30 in the main tests). We report 95 % confidence intervals by resampling clusters, and paired Wilcoxon tests with Holm correction. None of the 12 related papers reports confidence intervals.

Step 11 — Testing fairly



Every block is a separate range of random seeds, so no test traffic is ever seen while training or choosing settings. Settings are chosen on validation seeds only; each test range is used exactly once, after the choice is committed to git.

Figure 6. Seed ranges used for each purpose.

- **Pre-registration.** Before each experiment, the settings to try, the rule for choosing, the hypotheses and the test seeds are written to a JSON file, hashed and committed to git.
- **Validation-only selection.** Settings are chosen on validation seeds only.
- **One-shot test.** The chosen settings are run once on fresh seeds; the code refuses to run a test twice.
- **Same treatment for baselines.** DMS-BTT got the same tuning budget, plus a blocklist variant, so we compare against its strongest version.

Where each step lives in the code

Step	File	Role
2	loftbench/config.py	Domains, attack variants, simulator settings
2	loftbench/traffic.py, ports.py	Packet-level traffic, labels, ingress ports
3	loftbench/sim.py	Bounded flow-table switch with a polling controller
4–5	loftbench/features.py	17 causal features from flow statistics
6–7	loftbench/models.py	Neural network, FedAvg/FedProx, calibration
8–9	loftbench/defenses.py	DualGuard, PortGuard, DMS-BTT, OVS evict, threshold-LRU
10	loftbench/revision1.py	Harm metric, cluster bootstrap, cluster Wilcoxon
11	loftbench/v21.py, revision2–4.py	Pre-registration, selection, one-shot tests
—	testbed/	Ryu app, Mininet topology and traffic drivers (not yet run)
—	tests/test_loftbench.py	21 unit tests pinning the simulator's behaviour

Results so far

Defense (fresh test seeds)	Normal flows harmed	Notes
No defense	about 42 %	The table is full most of the attack
DMS-BTT, tuned + blocklist (strongest baseline)	4.5 %	Better on its own attack settings; weak on mimicry (A4) and the unseen A6
Each controller learning alone	7.9 %	Misses variants seen only elsewhere
FL-DualGuard v2.1 (federated)	1.5 %	Matches pooling all data; 2.9× fewer harmed than DMS-BTT
Fast attacker (refresh ≤ 1 s): v2.1 alone	72 %	The detector does not recognise it
Fast attacker: PortGuard alone	6.2 %	Bounded at every speed when attackers have their own ports; fails if they control every port
Fast attacker: v2.1 + PortGuard	36 %	The two conflict; step 4b will fix the composition

Still to do before submission: run the Mininet/Ryu testbed with real background traces, add the standard comparison table and classifier comparison, and add the rDefender, FTGuard, SIFT and TableGuard baselines (see `docs/experiments_from_related_work.md`).